

SPRING

What is Spring??

Spring framework is an open source lightweight framework used for developing Java applications.

Before spring, in plain java we write code like this:

```
CoffeeRepository repo = new CoffeeRepository();  
CoffeeService service = new CoffeeService(repo);
```

problems:

- you are creating objects manually
- tight coupling
- hard to test
- hard to scale

Spring is a framework that creates, manages and connects objects for you. You don't create objects. Spring creates them and injects them where needed.

Core Concepts :

1. Inversion of Control (IoC):

IOC is a design principle where control of object creation and lifecycle is managed by the framework/container rather than by the developer.

Normally:

You control object creation

With Spring:

Spring controls object creation

So control is inverted → hence Inversion of Control.

IOC: programmer will focus on business logic and object creation will be given to spring framework.

It is responsible for creating, configuring and managing lifecycle of objects called beans.

2. Dependency Injection (DI):

DI is HOW IoC is implemented.

DI is a design pattern which allows objects to be injected with their dependencies rather than creating them themselves.

Instead of this:

```
CoffeeService service = new CoffeeService();
```

Spring does this:

```
@Autowired
```

```
CoffeeService service;
```

Meaning:

“Inject the dependency for me.”

3. Bean:

A Bean is simply:

An object managed by Spring IoC Container.

If Spring creates it → it's a bean.

If you create it using new → it's NOT a bean.

Spring Beans:

Defined using @Component, @Service, @Repository.

4. IoC Container:

This is Spring's brain.

It:

Creates beans

Manages lifecycle

Injects dependencies

Two types:

BeanFactory (basic, lightweight, lazy)

ApplicationContext (advanced, commonly used for enterprise)

Spring XML Config:

XML configuration was commonly used to define beans (objects) and dependencies in an XML file instead of annotations.

Spring MVC:

Framework for building web applications using Spring framework.

This framework is developed around a DispatcherServlet which dispatches requests to handlers.

(Serves as front controller)

Model – Contains application data

View – representation of delivered info in certain format

Controller – houses application business logic

((@Controller used to identify class as controller)

Spring provides @ExceptionHandler and @ControllerAdvice for handling exceptions globally.

@ControllerAdvice → Global exception handling for all controllers

@ExceptionHandler → handles specific exceptions in controller

Spring Framework vs Spring Boot

Problem with Spring Framework (before Boot)

Using Spring alone:

Too much configuration

XML / Java config everywhere

External Tomcat needed

Too many dependencies to manage

Slow project setup

Developers spent more time configuring than coding.

Spring Boot

Spring Boot is a Java based framework built on Spring that simplifies development.

Provides helper classes and annotations.

Auto-configuration

Dependency management

Embedded servers

We can use embedded web servers such as Tomcat, Jetty.

No need to install any server separately.

Auto-configuration → automatically configures based on:
what libraries are present
what you didn't configure yourself

Starter dependencies → no dependency hell

Example:
spring-boot-starter-web this includes all..

Spring Framework:

Install Tomcat
Deploy WAR file

Spring Boot:

Tomcat comes inside the app
Run as JAR

In Spring Framework, we deploy WAR files to an external Tomcat server.

In Spring Boot, Tomcat is embedded inside the application, so we run it as a JAR file.

First: What is Tomcat?

Tomcat is a web server.

A web server is a program that receives requests from a browser and sends responses.

When you open:

<http://localhost:8080>

Tomcat:

listens on port 8080
runs your Java web application
sends HTML or JSON back to the browser

Without Tomcat, your web app cannot run.

Second: What is a JAR file?

JAR = Java ARchive

A ZIP-like file containing:

.class files
libraries
configuration

Used for:

Normal Java applications

Console applications

Spring Boot applications

Run using:

java -jar app.jar

Third: What is a WAR file?

WAR = Web ARchive

Specially made for web applications.

A WAR file does not run by itself.

It must be deployed inside a server like Tomcat.

Example:

myapp.war → deploy in Tomcat → app runs

SPRING ANNOTATIONS:

Annotations are metadata that provide information to the compiler, runtime, or frameworks.

@Component

Tells Spring to manage this class as a bean.

“Spring, please create an object of this class and keep it ready.”

Example

@Component

```
public class CalculatorService {  
    public int add(int a, int b) { return a + b; }  
}
```

📌 Now: Spring creates this object

This class becomes a Spring Bean

You should NOT create it using new.

@Service

Used for service layer classes.

— Same as `@Component`, but for business logic Technically: `@Service = @Component +` meaning Use it when the class: contains business logic acts as a service layer

Example

`@Service`

```
public class CoffeeService { public String getCoffee() { return "Cappuccino"; } } 💡
```

Interview tip: “`@Service` is a specialization of `@Component` used for service layer classes.”

@Repository

Used for database layer. Enables exception translation.

Used when the class: talks to DB stores or retrieves data

Example

`@Repository`

```
public class CoffeeRepository { public String findCoffee() { return "Espresso"; } }
```

@Autowired

Injects dependencies into bean at runtime.

@Configuration

Marks class as a source of bean definitions.

@Bean

Used in Java configuration to define a Spring bean.

@ComponentScan

Detects and registers beans in specified package.

@Value

Injects values from properties file or system properties.

@PropertySource

Loads external properties file into Spring environment.

@Primary

Specifies default bean when multiple candidates exist.

@Qualifier

Specifies which bean to inject when multiple beans of same type exist.(Qualifier overrides primary)

@Transactional

Ensures database operations are executed in a transaction.

What is @Component?

`@Component` is an annotation that tells Spring:

“Create an object of this class and manage it.”

When you put `@Component` on a class, Spring will:

- Detect it during component scanning
- Create its object
- Store it inside the IoC container
- Manage its lifecycle

What is @Bean?

`@Bean` is used inside a `@Configuration` class to manually define a bean.

Example:

```
@Configuration
public class AppConfig {

    @Bean

    public PaymentService paymentService() {

        return new PaymentService();

    }

}
```

BEAN SCOPES

Singleton (default) – one instance per spring container

Prototype – new instance each time

Request – one per HTTP request

Session – one per HTTP session

Application – one per ServletContext

Lifecycle of Spring Bean: 8steps

1. Instantiation – Bean is created (Constructor)
2. Dependency Injection – Dependencies injected
3. Post Initialization (`@PostConstruct`) – Custom initialization
4. Ready to use
5. Pre-Destroy (`@PreDestroy`) – Cleanup before destruction.

DI TYPES

When to Use Constructor Injection

Use when dependency is mandatory.

The class cannot work without it.

In your project:

ProductService cannot work without ProductRepository.

So we use:

```
public ProductService(ProductRepository repository) {  
    this.repository = repository;  
}
```

Think like this:

If ProductService is a car

Repository is the engine

A car cannot exist without an engine.

So the engine must be given at creation time.

Mandatory

Always required

Should not change

Use Constructor Injection.

When to Use Setter Injection

Use when dependency is optional.

Or when it may change later.

In your project (old version):

Search strategy:

```
setSearchStrategy(new NameSearchStrategy());
```

Strategy could change:

name search

category search

price search

So it was optional and dynamic.

Think:

A car can run without a music system.
Music system can be added later.

So we use setter injection.

Optional
Can change later
Not required for object creation
Use Setter Injection.

When to Use Field Injection

`@Autowired`
`private ProductRepository repository;`

Use only for:

Quick demo
Very small project
Testing prototype

☐ Not recommended for production.

Why?

Hard to test
Hidden dependency
Not immutable
Not clean architecture.

How to Create & Use a Bean:

Bean is an object managed by Spring IoC container.

Once created, it can be injected into another class using `@Autowired`.

Default bean name in `@Bean` is the method name.

Multiple names:

`@Bean(name = { "...", })`

Spring Data JPA

JPA (Java Persistence API) is an API that acts as a bridge between Java application and relational database.

Used to perform CRUD operations.

JDBC

Before JPA existed, we used JDBC. JDBC (Java Database Connectivity) is a low-level API that allows Java applications to interact with relational databases.

DriverManager?

DriverManager is responsible for establishing a connection between Java application and database using registered JDBC drivers.

Connection represents a session between Java application and database.

Once connection is established:

- You can execute queries
- You can commit/rollback transactions
- You can close the connection

```
Connection con = DriverManager.getConnection(...);
```

PreparedStatement

PreparedStatement is used to execute parameterized SQL queries.

ResultSet

ResultSet holds data returned from a SELECT query.

Problems with JDBC

1. Boilerplate code
2. Manual SQL writing
3. Manual object mapping
4. Manual transaction handling
5. No object graph support
6. No caching
7. Hard to maintain large applications

This pain led to ORM.

ORM (Object Relational Mapping):

ORM allows us to work with database using Java objects instead of writing SQL manually.

Java World	Database World
-------------------	-----------------------

Class	Table
-------	-------

Object	Row
--------	-----

Field	Column
-------	--------

Object Reference	Foreign Key
------------------	-------------

Collection	One-to-Many
------------	-------------

What ORM Solves

1. Automatic Object Mapping

You don't manually write:

```
user.setId(...)
```

ORM does it automatically.

2. Relationship Handling

Example:

```
class Order {  
    private User user;  
}
```

Database:

ORDER_TABLE

USER_ID (Foreign Key)

ORM automatically maps:

- User object ↔ USER_ID

3. Reduces Boilerplate Code

Instead of: PreparedStatement

ResultSet

Mapping

Closing resources

You write:

```
User user = entityManager.find(User.class, 1L);
```

4. Database Independence

You change:

- MySQL → PostgreSQL

ORM handles dialect changes internally.

5. Caching & Performance Optimization

Think like this:

Spring Data JPA

↓

JPA

↓

Hibernate

↓

JDBC

↓

Database

JPA

JPA is a specification that defines how Java objects should be mapped to relational databases. It provides interfaces and annotations but does not provide implementation.

It is NOT a framework.

It is NOT an implementation.

It is just a set of rules (interfaces + annotations).

Example things JPA defines:

- @Entity
- @Table
- @Id
- @OneToMany
- EntityManager
- PersistenceContext

But JPA does NOT implement anything.

It only says:

"Any ORM framework must follow these rules.

Hibernate

Hibernate is a framework that maps Java objects to relational database tables.

It is an implementation of JPA.

It:

- Generates SQL
- Connects using JDBC
- Manages caching
- Handles transactions
- Does dirty checking(**Dirty checking** is a mechanism used by **Hibernate** to determine whether any value of an entity has changed since it was retrieved from the database)

Spring Data JPA

Spring Data JPA is an abstraction layer on top of JPA.

It removes even more boilerplate.

What It Actually Does

Spring Data JPA:

- Creates repository implementation at runtime
- Uses JPA (EntityManager)
- Which uses Hibernate
- Which uses JDBC

Layer	Type	Role
JPA	Specification	Defines rules
Hibernate	Implementation	Implements JPA
Spring Data JPA	Abstraction	Simplifies JPA usage

- **ORM = concept**
- **Hibernate = ORM framework**

- **JPA = ORM specification**
- **Spring Data JPA = abstraction layer**

So:

Hibernate is an ORM framework.

JPA defines how ORM should work in Java.

CORE SPRING DATA JPA

Entity Mapping

1 @Entity

Marks a class as a JPA entity (persistent object).

@Entity

```
public class User {
}
```

Meaning:

This class should be mapped to a database table.

2 @Table

Defines table name and properties.

@Entity

@Table(name = "users")

```
public class User {
}
```

If you don't use it:

- Table name defaults to class name.

@Table(

name = "users",

uniqueConstraints = @UniqueConstraint(columnNames = "email")

)

3 @Id (Primary Key)

Every entity MUST have a primary key.

@Id

```
private Long id;
```

This uniquely identifies a row.

4. @GeneratedValue

Defines how primary key is generated.

@Id

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
private Long id;
```

Primary Key Generation Strategies

a) IDENTITY

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
```

Database auto-increments ID

b) SEQUENCE

```
@GeneratedValue(strategy = GenerationType.SEQUENCE)
```

Uses database sequence object

c) AUTO

```
@GeneratedValue(strategy = GenerationType.AUTO)
```

JPA chooses strategy automatically.

5. @Column

Used to customize column mapping.

```
@Column(
```

```
    name = "user_name",
```

```
    nullable = false,
```

```
    unique = true,
```

```
    length = 50
```

```
)
```

```
private String name;
```

Options:

- nullable
- unique

- length
- updatable
- insertable

If not specified:

- Defaults are used.
-

6 @Transient

Very important.

@Transient

private int age;

Means:

This field should NOT be stored in database.

Used for:

- Calculated values
 - Temporary data
-

7 @Enumerated

Used for enums.

@Enumerated(EnumType.STRING)

private OrderStatus status;

STRING vs ORDINAL

ORDINAL (Default)

@Enumerated(EnumType.ORDINAL)

Stored as number:

0

1

2

If enum changes order → DB becomes corrupted 😱

Example:

If you reorder enum values → old data becomes invalid.

 Dangerous.

STRING (Recommended)

`@Enumerated(EnumType.STRING)`

Stored as:

NEW

COMPLETED

- ✓ Safe
- ✓ Readable
- ✓ Stable

Always use STRING in real projects.

Complete Example Entity

`@Entity`

`@Table(name = "orders")`

`public class Order {`

`@Id`

`@GeneratedValue(strategy = GenerationType.IDENTITY)`

`private Long id;`

`@Column(nullable = false)`

`private String customerName;`

`@Enumerated(EnumType.STRING)`

`private OrderStatus status;`

@Lob

private String description;

@Transient

private double discount;

}

How This Looks in DB

ORDERS

ID (PK)

CUSTOMER_NAME

STATUS

DESCRIPTION

discount not stored.

Repository Layer

Repository layer is responsible for Communicating with the database.

1 CrudRepository

This is the base interface.

public interface CrudRepository<T, ID>

Provides basic CRUD operations.

Main methods:

- save()
- findById()
- findAll()
- delete()
- count()
- existsById()

❷ PagingAndSortingRepository

Extends CrudRepository. {Pagination means:

Fetching data in small chunks (pages) instead of loading the entire table at once.}

Adds:

- Pagination
- Sorting

Methods like:

findAll(Pageable pageable)

findAll(Sort sort)

❸ JpaRepository (Most Used)

Extends:

- PagingAndSortingRepository
- CrudRepository

Adds:

- Batch operations
- flush()
- deleteInBatch()
- saveAndFlush()

This is what we usually use.

```
public interface UserRepository extends JpaRepository<User, Long>
```

Common Methods

❶ save()

```
userRepository.save(user);
```

What it does:

- If ID is null → INSERT
- If ID exists → UPDATE

❷ findById()

```
Optional<User> user = userRepository.findById(1L);
```

Returns Optional.

1L is a long literal. This number is of type long.

Internally:

- Uses EntityManager.find()
- First checks persistence context (1st level cache)
- If not found → queries database



Important:

Always handle Optional properly.

3 findAll()

```
List<User> users = userRepository.findAll();
```

Returns all rows.

Internally:

- Executes SELECT * FROM users
- Maps rows → objects

If pagination:

```
Page<User> page = userRepository.findAll(PageRequest.of(0, 10));
```

This prevents loading entire table.

4 delete()

```
userRepository.delete(user);
```

Or:

```
userRepository.deleteById(1L);
```

Internally:

- Loads entity
- Calls EntityManager.remove()
- Executes DELETE query

Derived Query Methods

What is a Derived Query in Spring Data JPA?

A **derived query** is:

A query that Spring automatically generates from the method name you write in the repository.

You do NOT write SQL.

You do NOT write JPQL.

You only write method name.

Spring “derives” the query from the method name.

Simple Example

Suppose you have this entity:

```
class Book {  
    Long id;  
    String title;  
}
```

If you write this in repository:

```
List<Book> findByTitle(String title);
```

You did NOT write SQL.

But Spring internally generates:

```
SELECT * FROM books WHERE title = ?
```

That is called a **Derived Query**.

Spring Data JPA can generate SQL **just by reading method names**

Spring splits method name into parts:

find + By + FieldName + Keyword

General Pattern:

<Action>By<Property><Operator>

❶ **findByName**

```
List<User> findByName(String name);
```

Generated JPQL(Java Persistence Query Language):

```
SELECT u FROM User u WHERE u.name = name
```

2 findByPriceGreaterThan

List<Product> findByPriceGreaterThan(Double price);

JPQL:

WHERE price > :price

Keywords:

- GreaterThan
 - LessThan
 - GreaterThanEqual
 - LessThanEqual
-

3 findByNameAndCategory

List<Product> findByNameAndCategory(String name, String category);

JPQL:

WHERE name = :name AND category = :category

Supported:

- And
 - Or
-

4 findByAgeBetween

List<User> findByAgeBetween(int start, int end);

JPQL:

WHERE age BETWEEN :start AND :end

5 findByEmailsIsNull

List<User> findByEmailsIsNull();

JPQL:

WHERE email IS NULL

Other useful keywords:

- IsNotNull

- Like
 - StartingWith
 - EndingWith
 - Containing
 - In
 - NotIn
 - OrderByNameAsc
 - OrderByPriceDesc
-

Important Rules

- Property names must match entity fields exactly.
- Case sensitive after By.
- Nested properties supported:

findByAddressCity(String city);

If:

```
class User {  
    Address address;  
}
```

Interview Tip

If query becomes complex → avoid long method names.

Use @Query.

@Query Annotation

Used when:

- Derived method is too complex
-

JPQL (Java Persistence Query Language)

JPQL works with:

- Entity names
- Field names

NOT table names.

Example:

```
@Query("SELECT u FROM User u WHERE u.name = :name")  
List<User> findUserByName(@Param("name") String name);
```

2 Native Query

If you want pure SQL:

```
@Query(value = "SELECT * FROM users WHERE name = :name", nativeQuery = true)  
List<User> findUserNative(@Param("name") String name);
```

Here:

- users = table name
 - name = column name
-

Named Parameters

Safer and readable.

```
@Query("SELECT u FROM User u WHERE u.age > :age")  
List<User> findUsers(@Param("age") int age);  
:age binds to method parameter.
```

Positional Parameters

Less readable.

```
@Query("SELECT u FROM User u WHERE u.age > ?1")  
List<User> findUsers(int age);  
?1 = first parameter  
?2 = second parameter
```

When To Use What?

Use Derived Methods:

- Simple filters
- Quick queries

Use JPQL:

- Complex joins
- Custom projections

Use Native Query:

- Database-specific functions
- Performance tuning
- Stored procedures

5 Relationships

◆ First: What Are Relationships in JPA?

In databases, relationships are maintained using **foreign keys**.

In Java, relationships are maintained using **object references**.

JPA connects these two worlds.

Example:

If many Orders belong to one User:

Database:

USERS

id

ORDERS

id

user_id ← foreign key

In Java:

```
class Order {
```

```
    User user;
```

```
}
```

That object reference (User user) must be mapped to user_id column.

That mapping is what relationship annotations do.

❶ @ManyToOne

Meaning:

Many records of this entity belong to one record of another entity.

Example:

Many Orders → One User

@Entity

```
class Order {
```

```
    @Id
```

```
    private Long id;
```

```
    @ManyToOne
```

```
    @JoinColumn(name = "user_id")
```

```
    private User user;
```

```
}
```

Explanation:

- @ManyToOne tells JPA: many orders belong to one user.
- @JoinColumn tells JPA: store foreign key in this table.
- user_id column is created in orders table.

Very important:

The side with @JoinColumn is the **owning side**.

❷ @OneToMany

Opposite of ManyToOne.

One User → Many Orders.

@Entity

```
class User {
```

@Id

private Long id;

@OneToMany(mappedBy = "user")

private List<Order> orders;

}

Explanation:

- mappedBy = "user" refers to the field name in Order class.
- It means: “The relationship is already managed by Order.user.”
- This side does NOT create foreign key.

Without mappedBy:

JPA creates an extra join table (which is usually wrong).

◆ Owning Side vs Inverse Side

This is VERY important.

Owning side:

- Contains foreign key.
- **Has @JoinColumn.**
- Controls the relationship.

Inverse side:

- **Uses mappedBy.**
- Does not control relationship.

In our example:

Entity Role

Order Owning side

User Inverse side

Only the owning side updates the database relationship.

3 @OneToOne

One entity maps to exactly one entity.

Example:

One User $\rightarrow$ One Profile

@Entity

```
class Profile {
```

```
    @Id
```

```
    private Long id;
```

```
    @OneToOne
```

```
    @JoinColumn(name = "user_id")
```

```
    private User user;
```

```
}
```

Database:

PROFILE

id

user_id (unique foreign key)

Important:

Foreign key must be unique to maintain one-to-one.

4 @ManyToMany

Many Students $\rightarrow$ Many Courses

Database:

STUDENT

COURSE

STUDENT_COURSE (join table)

Java:

@Entity

```
class Student {
```

```
    @ManyToMany
```

```

@JoinTable(
    name = "student_course",
    joinColumns = @JoinColumn(name = "student_id"),
    inverseJoinColumns = @JoinColumn(name = "course_id")
)

private List<Course> courses;
}

```

Explanation:

- JPA creates a join table.
- No foreign key directly in student or course.
- Relationship stored in middle table.

◆ FetchType (LAZY vs EAGER)

This controls **when related data is loaded**.

EAGER

Related entity loads immediately.

Example:

Load Order → User loads automatically.

Problem:

May load unnecessary data.

Can cause performance issues.

LAZY

Related entity loads only when accessed.

Example:

```
user.getOrders();
```

Only then query runs.

Better for performance.

Default Fetch Types:

- **ManyToOne** → **EAGER**
- **OneToOne** → **EAGER**
- **OneToMany** → **LAZY**
- **ManyToMany** → **LAZY**

Best practice:

Prefer LAZY unless absolutely necessary.

◆ **Cascade Types**

cascade types determine how operations propagate between related entities.

Cascade means:

What happens to child when parent is saved/deleted?

Example:

```
@OneToMany(mappedBy = "user", cascade = CascadeType.ALL)
```

If you save user:

```
userRepository.save(user);
```

Orders also get saved automatically.

Main types:

- **PERSIST** → save child
- **MERGE** → update child
- **REMOVE** → delete child
- **ALL** → everything

Without cascade:

You must save child manually.

◆ **mappedBy (Deep Understanding)**

mappedBy tells JPA:

"This side is NOT responsible for managing the relationship."

It must match the variable name in the owning entity.

Example:

```
@OneToMany(mappedBy = "user")
```

"user" refers to:

@ManyToOne

private User user;

If incorrect name → error.

1 Pagination & Sorting (Spring Data JPA)

1. Why Pagination is Important

Problem Without Pagination

If you call:

```
repository.findAll();
```

Hibernate generates:

```
SELECT * FROM table;
```

Issues:

- Loads entire table into memory
- High RAM usage
- Slow response time
- Not scalable

In production systems with millions of rows → this is unacceptable.

Pagination ensures:

- Controlled memory usage
 - Faster response
 - Database-level slicing
-

2. Pageable

What is Pageable?

Pageable is an interface that holds:

- Page number
- Page size
- Sorting details

It does NOT execute queries.
It only describes how data should be fetched.

Creating Pageable

Pageable pageable = PageRequest.of(pageNumber, pageSize);

Example:

PageRequest.of(2, 20);

Meaning:

- Page index = 2 (0-based)
 - Page size = 20
 - Offset = $2 \times 20 = 40$
-

SQL Generated

SELECT * FROM table

LIMIT 20 OFFSET 40;

Important:

- Pagination happens at **database level**
 - Hibernate does NOT paginate in memory
-

3. Page

What is Page?

Page<T> is a result wrapper that contains:

- Content (list of entities)
 - Total elements
 - Total pages
 - Current page number
-

Internally Executes Two Queries

❶ Data Query


```
SELECT * FROM table
```

```
LIMIT ? OFFSET ?;
```

2 Count Query

```
SELECT COUNT(*) FROM table;
```

Count query is needed to calculate total pages.

When to Use Page

- Admin dashboards
 - UI with page numbers
 - When total count is required
-

4. Slice

What is Slice?

Slice<T> is lighter than Page.

It:

- Executes only data query
- Does NOT execute count query
- Only checks if next page exists

Internally fetches one extra row to determine next page.

When to Use Slice

- Infinite scrolling
 - APIs where total count is unnecessary
 - Performance-sensitive endpoints
-

5. Page vs Slice

Feature	Page	Slice
Executes count query	Yes	No

Feature	Page	Slice
Knows total pages	Yes	No
Performance	Slightly slower	Faster
Best for	Pagination UI	Infinite scroll

6. Sorting

Sorting defines result order at database level.

Example:

```
Sort sort = Sort.by("price").descending();
```

SQL:

```
ORDER BY price DESC
```

Combining Pagination & Sorting

```
PageRequest.of(0, 10, Sort.by("createdAt").descending());
```

SQL:

```
SELECT * FROM table
```

```
ORDER BY created_at DESC
```

```
LIMIT 10 OFFSET 0;
```

8. Common Interview Questions

Q1: How does pagination work internally?

Spring Data JPA converts Pageable into SQL using LIMIT and OFFSET.
Page executes an additional COUNT query.

Q2: Difference between Page and Slice?

Page executes count query and provides total pages.
Slice skips count query and only checks if next page exists.

Q3: Where does pagination happen?

At database level, not in memory.

Q4: What is default page index?

0 (zero-based indexing).

Specifications

Specifications allow dynamic construction of WHERE clauses using JPA Criteria API. They are used in enterprise applications where filtering conditions are optional and combinable. By extending JpaSpecificationExecutor, we can compose reusable query predicates and combine them dynamically using AND/OR conditions.

Imagine you are building an e-commerce search API.

Users can filter products by:

- name (optional)
- category (optional)
- minimum price (optional)
- maximum price (optional)

Example request:

/products?name=phone&category=electronics&minPrice=10000

But sometimes user may send:

/products?category=electronics

Or:

/products?minPrice=5000&maxPrice=20000

All filters are optional.

Why Derived Query Methods Fail Here

With derived methods, you would need:

- findByName()
- findByCategory()
- findByNameAndCategory()
- findByNameAndPriceBetween()

- `findByCategoryAndPriceBetween()`
- `findByNameAndCategoryAndPriceBetween()`

This becomes impossible to maintain.

In real enterprise apps, filters grow to 10–15 parameters.

That's why Specifications exist.

2 What is Specification?

A Specification is:

A reusable piece of query logic that represents a single filtering condition.

Internally, it uses JPA Criteria API.

It builds SQL dynamically at runtime.

Think of it like:

- Small condition blocks
 - That can be combined dynamically
-

3 What is JpaSpecificationExecutor?

Normally, JpaRepository does not support Specifications.

To enable dynamic queries, your repository must extend:

`JpaSpecificationExecutor<Product>`

Example:

```
public interface ProductRepository extends
    JpaRepository<Product, Long>,
    JpaSpecificationExecutor<Product> {
}
```

This gives you:

`findAll(Specification spec)`

`findAll(Specification spec, Pageable pageable)`

Projections

📌 1 Why Projections Are Needed

♦ The Real Problem

Suppose you have this entity:

@Entity

```
class User {
```

```
    @Id
```

```
    private Long id;
```

```
    private String name;
```

```
    private String email;
```

```
    private String password;
```

```
    private String address;
```

```
}
```

Now your API only needs:

- name
- email

But if you write:

```
List<User> users = userRepository.findAll();
```

Hibernate executes:

```
SELECT * FROM users;
```

This fetches:

- id
- name
- email
- password
- address

Even though you only need 2 fields.

- ✗ Unnecessary data
 - ✗ More memory usage
 - ✗ Slower performance
 - ✗ Security risk (password exposed)
-

✓ **Solution: Projection**

Projection means:

Fetch only required fields instead of the entire entity.

Instead of:

```
SELECT *
```

We do:

```
SELECT name, email
```

This improves performance.

📌 **2 Types of Projections**

Spring Data JPA supports:

- 1 Interface-Based Projection
 - 2 DTO Projection
 - 3 Constructor Projection (JPQL)
-

1 **Interface-Based Projection**

What Is It?

You create an interface with only required getters.

Example:

```
public interface UserView {  
    String getName();  
    String getEmail();  
}
```

Then in repository:

```
List<UserView> findByAgeGreaterThan(int age);
```

Spring automatically:

- Fetches only required columns
- Creates proxy object implementing UserView

Generated SQL:

```
SELECT name, email FROM users WHERE age > ?;
```

When To Use?

- ✓ Simple read operations
 - ✓ When only few fields are needed
 - ✓ Quick and easy solution
-

2 DTO Projection

What Is DTO?

DTO = Data Transfer Object

It is a simple class used to transfer data.

Example:

```
public class UserDTO {  
  
    private String name;  
    private String email;  
  
    public UserDTO(String name, String email) {  
        this.name = name;  
        this.email = email;  
    }  
  
    // getters  
}
```

DTO:

- Is NOT an entity
 - Is NOT managed by JPA
 - Just holds data
-

Using DTO in Repository

```
@Query("SELECT new com.example.UserDTO(u.name, u.email) FROM User u")
```

```
List<UserDTO> findUsers();
```

Important:

```
new com.example.UserDTO(...)
```

This tells Hibernate:

- Create DTO directly
 - Do NOT create entity
 - Do NOT track entity
-

3 Constructor Projection (JPQL Constructor Expression)

This is how DTO projection works internally.

Using:

```
SELECT new com.example.UserDTO(u.name, u.email)
```

Hibernate:

- Fetches only specified fields
- Calls DTO constructor
- Returns DTO objects

No entity is loaded.

 **Spring Data JPA DOES support constructor projection.**

But it supports it through **JPQL constructor expressions**, not automatically like interface projections.

3 Entity vs Projection (Important for Interview)

Entity	Projection
Fetches all fields	Fetches selected fields
Managed by Hibernate	Not managed
Uses persistence context	No tracking
Heavy	Lightweight
Used for update operations	Used for read APIs

Common Interview Questions

Q1: What is projection in Spring Data JPA?

Projection allows fetching only selected fields instead of loading the entire entity.

Q2: Difference between interface projection and DTO projection?

Interface projection uses dynamic proxy.

DTO projection uses constructor expression in JPQL.

Q3: Why is projection better for performance?

Because it avoids entity loading and persistence context tracking.

Final Short Interview Summary

Projections in Spring Data JPA are used to fetch only required fields instead of loading full entities. They improve performance by reducing memory usage and avoiding persistence context tracking. Spring supports interface-based projections and DTO constructor projections.

1 What is **Transaction?**

A transaction is:

A group of database operations that must either fully succeed or fully fail.

Example:

Money transfer:

1. Deduct ₹1000 from Account A
2. Add ₹1000 to Account B

If deduction succeeds but addition fails → system is corrupted ❌

So both operations must succeed together.

That is a transaction.

🔵 2 What is @Transactional?

@Transactional tells Spring:

“Run this method inside a database transaction.”

Example:

@Transactional

```
public void transferMoney() {  
    debit();  
    credit();  
}
```

If anything fails → rollback

If everything succeeds → commit

🔵 3 How Transaction Works Internally (Very Important)

This is what interviewers love.

When you add:

@Transactional

Spring does NOT modify your method.

Instead:

Step-by-step internal flow:

- 1 Spring creates a **proxy** for your service class.
- 2 When method is called → proxy intercepts call.
- 3 Proxy starts a transaction.
- 4 Method executes.

- 5] If no exception → commit transaction.
- 6] If runtime exception → rollback transaction.

So:

Client → Proxy → Transaction Manager → Hibernate → JDBC → Database

A proxy is a **middle layer** between you and the real object.

4] Propagation Types

Propagation defines:

What happens if a transactional method calls another transactional method?

Default = REQUIRED

REQUIRED (Default)

Meaning:

- If transaction exists → join it
- If not → create new

Most common and safe.

REQUIRES_NEW

Meaning:

- Suspend current transaction
- Create completely new transaction

Used for:

- Logging
 - Audit records
 - Independent save operations
-

SUPPORTS

- If transaction exists → join
- If not → run without transaction

✓ MANDATORY

- Must have existing transaction
- Else → exception

✓ NEVER

- Must NOT run inside transaction
- Else → exception

🔵 7 Important Real Interview Concepts

? Why doesn't @Transactional work on private methods?

Because Spring uses proxies.

Proxy can intercept only public methods.

? Why doesn't it work in self-invocation?

If method inside same class calls another method,

Proxy is bypassed.

Transaction won't apply.

? When does rollback happen?

By default:

- On RuntimeException
- Not on checked exceptions (unless configured)

Entity Lifecycle

In JPA, an entity goes through different stages:

- Created
- Saved
- Updated

- Loaded
- Deleted

JPA allows you to run methods **automatically** when these events happen.

Those automatic methods are called **lifecycle callbacks**.

You define them inside your entity class.

@PrePersist

When does it run?

Before the entity is inserted into the database.

Meaning:

Before SQL INSERT happens.

Example

@Entity

class User {

 @Id

 private Long id;

 private LocalDateTime createdAt;

 @PrePersist

 public void setCreatedAt() {

 this.createdAt = LocalDateTime.now();

 }

}

@PreUpdate

When does it run?

Before an existing entity is updated in the database.

Before SQL UPDATE happens.

✓ Example

@PreUpdate

```
public void setUpdatedAt() {  
    this.updatedAt = LocalDateTime.now();  
}
```

● @PostLoad

✓ When does it run?

After entity is fetched from database.

After SQL SELECT completes.

✓ Example

@PostLoad

```
public void afterLoad() {  
    System.out.println("User loaded from DB");  
}
```

● @PreRemove

✓ When does it run?

Before entity is deleted from database.

Before SQL DELETE happens.

✓ Example

@PreRemove

```
public void beforeDelete() {  
    System.out.println("User will be deleted");  
}
```

🔵 6 Full Example Entity

@Entity

class User {

 @Id

 private Long id;

 private LocalDateTime createdAt;

 private LocalDateTime updatedAt;

 @PrePersist

 public void prePersist() {

 createdAt = LocalDateTime.now();

 }

 @PreUpdate

 public void preUpdate() {

 updatedAt = LocalDateTime.now();

 }

 @PostLoad

 public void postLoad() {

 System.out.println("Entity loaded");

 }

 @PreRemove

 public void preRemove() {

 System.out.println("About to delete");

```
}  
  
}
```

🔵 7 Important Interview Points

- ✓ Lifecycle methods are inside entity class
 - ✓ They run automatically
 - ✓ They run inside transaction
 - ✓ They are used mainly for auditing
 - ✓ They must not contain heavy business logic
-

🔵 8 Quick Revision Table

Annotation Runs When

@PrePersist Before INSERT

@PreUpdate Before UPDATE

@PostLoad After SELECT

@PreRemove Before DELETE

🔵 9 One-Line Interview Definition

Entity lifecycle callbacks in JPA allow execution of methods automatically before or after database operations like insert, update, load, and delete.

🔥 1 N+1 Problem in JPA / Hibernate

💡 What is N+1 Problem?

The **N+1 problem** happens when:

- 1 query fetches a list of parent entities
- Then N additional queries fetch child entities one by one

So total queries =

👉 **1 + N queries**

That's why it is called **N+1**.

Real Example (E-Commerce Scenario – Like our Project)

Let's say:

- One Order
- Each order has multiple OrderItems

Entities:

@Entity

```
class Order {
```

```
    @OneToMany(mappedBy = "order", fetch = FetchType.LAZY)
```

```
    private List<OrderItem> items;
```

```
}
```

@Entity

```
class OrderItem {
```

```
    @ManyToOne
```

```
    private Order order;
```

```
}
```

What Happens Internally?

You write:

```
List<Order> orders = orderRepository.findAll();
```

Query 1:

```
SELECT * FROM orders;
```

Then inside loop:

```
for (Order o : orders) {
```

```
    o.getItems().size();
```

```
}
```

Now Hibernate fires:

```
SELECT * FROM order_items WHERE order_id = 1;
```

```
SELECT * FROM order_items WHERE order_id = 2;
```

```
SELECT * FROM order_items WHERE order_id = 3;
```

...

If 100 orders →

👉 1 + 100 queries 😱

That is the N+1 problem.

? Why It Happens?

It happens mainly because of:

♦ LAZY Loading (Default in OneToMany)

Hibernate loads related entities only when accessed.

So:

- First query loads parent
 - Accessing child triggers separate query
-

♦ Even EAGER Can Cause N+1

Even if you use:

```
fetch = FetchType.EAGER
```

Hibernate may still fire multiple queries depending on relationship and JPQL.

So EAGER is NOT the solution.

🚨 Why It Is Dangerous?

In production:

If:

- 1000 orders
- Each order has 5 items

You may execute **1001 queries**

This causes:

- Slow API

- High DB load
 - Performance degradation
 - Timeouts
-

✅ How To Solve N+1 Problem

The N+1 problem occurs when Hibernate executes one query to fetch parent entities and then executes additional queries for each associated child entity due to lazy loading. This leads to performance issues. It can be solved using JPQL fetch join, Spring's `@EntityGraph`, or Hibernate batch fetching configuration.

🔥 1 Fetch Join (BEST & Most Used)

Use JPQL with JOIN FETCH.

```
@Query("SELECT o FROM Order o JOIN FETCH o.items")
```

```
List<Order> findAllWithItems();
```

What happens?

Hibernate runs:

```
SELECT *
```

```
FROM orders o
```

```
JOIN order_items i ON o.id = i.order_id;
```

- 👉 Only ONE query
 - 👉 Loads everything in single go
 - 👉 N+1 solved
-

⚠ Important

Fetch join works only in JPQL, not in normal `findAll()`

🔥 2 @EntityGraph (Cleaner Spring Way)

Instead of writing custom query:

```
@EntityGraph(attributePaths = {"items"})
```

```
List<Order> findAll();
```

This tells Hibernate:

👉 "Fetch items along with orders"

Under the hood → behaves like fetch join.

Why @EntityGraph is Good?

- Cleaner
 - Declarative
 - Works well with Spring Data JPA
 - Less error-prone
-

🔥 3 Batch Size (Partial Optimization)

If you cannot use fetch join, use batching.

@OneToMany(mappedBy = "order")

@BatchSize(size = 10)

private List<OrderItem> items;

Or global:

spring.jpa.properties.hibernate.default_batch_fetch_size=10

What Happens?

Instead of:

SELECT * FROM order_items WHERE order_id = 1;

SELECT * FROM order_items WHERE order_id = 2;

SELECT * FROM order_items WHERE order_id = 3;

Hibernate does:

SELECT * FROM order_items

WHERE order_id IN (1,2,3,4,5,6,7,8,9,10);

👉 Reduces N queries

👉 Not fully 1 query

👉 But much better

Fetch join is generally the most efficient solution because it eliminates N+1 completely by executing a single SQL query. However, for paginated queries or very large datasets, batch fetching may be more appropriate.

❓ Fetch Join vs Lazy Loading (Deep Internal Explanation)

❓ Interviewer: What is Lazy Loading and how does it work internally?

Answer:

Lazy loading means related entities are not loaded immediately. Instead, Hibernate creates a **proxy object** and fetches the data only when the association is accessed.

Internally:

- Hibernate uses **Bytecode Enhancement or CGLIB proxy**
- When entity is loaded, associated entity is replaced with proxy
- When you access a method → proxy triggers SQL query

Example:

```
@OneToMany(fetch = FetchType.LAZY)
```

```
private List<Order> orders;
```

Flow:

1. User loaded
2. Orders NOT loaded
3. When user.getOrders() is called → SELECT query fires

Problem:

- Causes **N+1 query problem**

❓ What is Fetch Join?

Answer:

Fetch Join loads parent and child in **single SQL query using JOIN**.

Example:

```
@Query("SELECT u FROM User u JOIN FETCH u.orders")
```

Generated SQL:

```
SELECT * FROM user u
```

JOIN orders o ON u.id = o.user_id;

- ✓ Avoids N+1 problem
 - ✓ Better performance
 - ✗ Can cause duplicate rows (needs DISTINCT)
-

💡 Senior-level Summary:

Lazy loading = Proxy-based delayed loading

Fetch Join = Immediate loading using JOIN

Choose based on use-case and query design

③ Caching — First-Level, Second-Level, Hibernate Cache

? What is First-Level Cache?

Answer:

First-level cache is **Persistence Context**.

Scope:

- Per Hibernate Session
- Always enabled

Internally:

- Uses `HashMap<EntityId, EntityObject>`

If you call:

`find(User.class, 1);`

`find(User.class, 1);`

Second call → NO SQL → returned from memory

? What is Second-Level Cache?

Answer:

Shared across sessions.

Stored in:

- EhCache
- Hazelcast

- Redis

Must enable manually:

```
spring.jpa.properties.hibernate.cache.use_second_level_cache=true
```

Flow:

1. First query → DB → stored in L2 cache
 2. Next session → fetch from cache
-

? What is Hibernate Cache?

Hibernate has:

- First-level (default)
 - Second-level (optional)
 - Query cache (optional)
-

💡 Interview Punchline:

First-level cache = Persistence context

Second-level cache = Shared across sessions

Query cache = Caches result sets

4 Dirty Checking (Very Important)

? How does JPA update entity without calling update()?

Answer:

JPA uses **Dirty Checking mechanism**.

Internally:

1. When entity loaded → snapshot stored in persistence context
2. On transaction commit → Hibernate compares snapshot vs current state
3. If changed → generates UPDATE query automatically

Example:

```
User user = repo.findById(1);
```

```
user.setName("Sweety");
```

No save() required inside transaction.

At commit → UPDATE fires.

Internal Mechanism:

Hibernate tracks:

- Original state
- Current state

Uses:

- Bytecode enhancement
 - Reflection
-

5 Optimistic vs Pessimistic Locking

What is Optimistic Locking?

Answer:

Uses version field.

@Version

private Long version;

Flow:

1. Entity read with version=1
2. Two users modify
3. First update → version becomes 2
4. Second update fails (OptimisticLockException)

No DB lock.

- ✓ High performance
 - ✓ Suitable for read-heavy systems
-

What is Pessimistic Locking?

Locks row in DB.

entityManager.lock(entity, LockModeType.PESSIMISTIC_WRITE);

SQL:

SELECT ... FOR UPDATE;

✓ Prevents concurrent access

✗ Slower

💡 Interview Comparison:

Optimistic → Version based

Pessimistic → DB row lock

Spring REST

What is REST?

REST (Representational State Transfer) is an architectural style used to design distributed systems, especially web services, where:

- Everything is treated as a **resource**
- Resources are accessed using **standard HTTP methods**
- Communication is **stateless**

REST was introduced by **Roy Fielding** in 2000.

REST is NOT

- ✗ Not a protocol
- ✗ Not a framework
- ✗ Not specific to Spring
- ✗ Not just @RestController

REST is a **design principle**.

Breaking Down the Term “**Representational State Transfer**”

♦ Resource

A resource is anything that can be identified using a URI. A **URI (Uniform Resource Identifier)** is a string used to **identify a resource** on the internet.

Examples:

/users

/users/10

/orders/25

/products

In REST:

Everything is a resource.

◆ Representation

The format in which the resource data is transferred.

Common formats:

- JSON
- XML

Example:

Request:

GET /users/10

Response:

```
{  
  "id": 10,  
  "name": "Prasanna"  
}
```

The JSON above is the **representation** of the resource.

◆ State Transfer

Each request transfers the current state of a resource.

Example:

POST /orders

This changes server state by creating a new order.

Important REST Concepts

Idempotency

An operation is idempotent if repeating it gives the same result.

Example:

PUT /users/10

Calling it multiple times → same result.

POST is not idempotent.

Safe Methods

Safe methods do not change server state.

Safe:

- GET

Not Safe:

- POST
- PUT
- DELETE

Key Principles of REST

A system is RESTful only if it follows these constraints.

2.1 Uniform Interface

Definition

REST enforces a consistent interface between client and server.

This simplifies architecture and improves visibility.

Uniform Interface Has 4 Parts

1 Identification of Resources

Each resource must have a unique URI.

Example:

/users

/users/10

/orders/25

❷ Manipulation Through Representations

Resources are not directly transferred.

Instead, their **representation** is transferred.

Example:

GET /users/10

Response:

```
{  
  "id": 10,  
  "name": "Prasanna"  
}
```

The JSON is the representation of the resource.

❸ Self-Descriptive Messages

Each response should contain enough information for the client to understand:

- What data is returned
- How to process it
- What actions are available next

Headers + media types help achieve this.

❹ Hypermedia (HATEOAS) (Hypermedia as the Engine of Application State)

Hypermedia = Links inside responses that guide client to next action.

HATEOAS is a REST constraint which states that:

A client should interact with a REST API entirely through hyperlinks provided dynamically by the server responses.

Example:

```
{  
  "id": 123,  
  "title": "What is REST",  
  "author": {
```

```
"id": 456,  
"profile_url": "https://example.com/authors/456"  
},  
"self": {  
  "link": "https://example.com/posts/123"  
}  
}
```

Client discovers next resources dynamically.

2.2 Client–Server Separation

Definition

Client and server are independent systems.

Client:

- UI
- Mobile app
- Browser

Server:

- Business logic
 - Database
 - Authentication
-

Why Important?

- Independent evolution
 - Scalability
 - Clear separation of concerns
-

2.3 Statelessness

Definition

Each request must contain all necessary information.

Server must NOT store client session state.

Example

Correct (Stateless):

GET /orders

Authorization: Bearer <JWT>

Server validates token and responds.

Benefits

- Easy scalability
 - No session management
 - Works well in distributed systems
-

2.4 Cacheable

Responses must declare whether they are cacheable and used later or not.

Example:

Cache-Control: max-age=3600

Improves performance and reduces server load.

2.5 Layered System

System may have multiple layers:

Client → API Gateway → Service → Database

Client does not know if it is connected to:

- Direct server
 - Load balancer
 - Proxy
 - Microservice
-

2.6 Code on Demand (Optional)

Server may send executable code to client (like JavaScript).

Rarely used in APIs.

7 Real World Example (E-Commerce)

Non-REST:

/createOrder

/getAllOrders

/deleteOrderById

RESTful:

POST /orders

GET /orders

GET /orders/{id}

DELETE /orders/{id}

4 REST and HTTP Are Not the Same

REST $\neq$ HTTP

REST is an architectural style.

HTTP is a protocol commonly used to implement REST.

You can build REST over other protocols.

5 HTTP Methods Commonly Used in REST

Method	Purpose
--------	---------

GET	Read resource
-----	---------------

POST	Create resource
------	-----------------

PUT	Update/Replace resource
-----	-------------------------

DELETE	Remove resource
--------	-----------------

PATCH	Partial update
-------	----------------

6 What is Spring REST?

6.1 Spring Framework Support for REST

Spring provides built-in support for building REST APIs.

Main components:

- Spring MVC
 - `@RestController`
 - `@RequestMapping`
 - `@GetMapping` / `@PostMapping`
 - `ResponseEntity`
 - Jackson (JSON conversion)
-

6.2 How Spring Makes REST Easy

Spring automatically:

- Maps HTTP requests to Java methods
 - Converts JSON to Java objects
 - Converts Java objects to JSON
 - Handles HTTP status codes
 - Supports validation
 - Integrates with security
-

6.3 Example in Spring Boot

`@RestController`

`@RequestMapping("/users")`

`public class UserController {`

`@GetMapping("/{id}")`

`public User getUser(@PathVariable int id) {`

`return new User(id, "Prasanna");`

`}`

`}`

Spring automatically:

- Converts User object to JSON
 - Returns HTTP 200 OK
 - Maps URI properly
-

7 REST vs RESTful Web Service

REST = Architectural style

RESTful Web Service = API that follows REST principles

If an API follows:

- Statelessness
- Uniform interface
- Client-server separation
- Cacheable
- Layered system

Then it is RESTful.

An **API (Application Programming Interface)** is a set of rules that allows two software applications to communicate with each other.

What @SpringBootApplication Does

It combines:

1 @Configuration

Marks class as configuration source.

2 @EnableAutoConfiguration

Automatically configures:

- Tomcat
- DispatcherServlet
- JSON converter

3 @ComponentScan

Scans components in same package.

spring-boot-starter-web – What It Provides

When building REST APIs in Spring Boot, we add:

spring-boot-starter-web

This single dependency brings all necessary components to build and run a web application.

1 Name: Spring MVC

Purpose:

Handles HTTP requests and maps them to controller methods. Returning responses

2 Name: Embedded Tomcat

Purpose:

Acts as the built-in web server. Running the application as a standalone JAR. No need for external Tomcat installation

3 Name: Jackson (JSON Processor)

Purpose:

Converts Java objects ↔ JSON automatically.

Used for:

- Serialization (Object → JSON)
- Deserialization (JSON → Object)

4 Name: REST Annotations

Examples:

- `@RestController`
- `@GetMapping`
- `@PostMapping`
- `@PutMapping`
- `@DeleteMapping`

Purpose:

Map HTTP methods to Java methods and define REST endpoints easily.

5 Name: DispatcherServlet

Purpose:

Acts as the **Front Controller** in Spring MVC.

It:

- Receives all incoming HTTP requests
- Finds matching controller
- Executes method
- Processes response
- Handles exceptions

Richardson Maturity Model (RMM)

The **Richardson Maturity Model (RMM)** is a model that measures how RESTful an API is.

It was introduced by **Leonard Richardson** and explained by **Martin Fowler**.

It defines **4 levels (0–3)** of REST maturity.

The higher the level, the closer the API follows REST principles.

2 Level 0 – HTTP as Transport (RPC Style)

Concept

- Uses HTTP only as a communication channel.
- Single endpoint.
- Usually only POST is used.
- Follows RPC (Remote Procedure Call) style.

Example

POST /appointmentService

Request body decides the operation.

Characteristics

- One URI
- No resource concept
- No proper HTTP method usage
- No proper status codes

Conclusion

This is **not RESTful**.
It is RPC over HTTP.

③ Level 1 – Resources

Concept

Introduce **resource-based URIs**.

Instead of one endpoint, use multiple resource identifiers.

Example

POST /doctors/mjones

POST /slots/1234

Each resource has a unique URI.

Characteristics

- Multiple URIs
 - Resource identification
 - Still may use only POST
 - No proper HTTP semantics yet
-

Benefit

Better structure and clarity.

Still not fully RESTful.

④ Level 2 – HTTP Verbs

Concept

Use HTTP methods properly.

Correct HTTP Methods

Method Purpose

GET	Read
POST	Create
PUT	Update
DELETE	Remove

Example

GET /doctors/mjones/slots
POST /slots/1234
DELETE /slots/1234

Use Proper HTTP Status Codes

Code Meaning

200	OK
201	Created
400	Bad Request
404	Not Found
409	Conflict

Characteristics

- Resource-based URIs
 - Proper HTTP verbs
 - Proper status codes
 - Enables caching
 - Safe vs unsafe method distinction
-

Importance

Most modern APIs are Level 2.

Often called **REST-like APIs**.

5 Level 3 – Hypermedia (HATEOAS)

Concept

Responses contain links that tell the client what actions are possible next.

Client discovers API dynamically.

Example

```
{  
  "id": "1234",  
  "_links": {  
    "self": { "href": "/slots/1234" },  
    "book": { "href": "/slots/1234/book" }  
  }  
}
```

Client does not hardcode URLs.

Characteristics

- Hypermedia controls in response
 - Discoverability
 - Self-descriptive responses
 - Loose coupling between client and server
-

Benefit

- Server can change URIs without breaking clients
 - API becomes evolvable
 - True REST compliance
-

6 Meaning of Levels

- Level 1 → Divide system into resources
- Level 2 → Standardize interaction using HTTP
- Level 3 → Add discoverability using hypermedia

Spring REST Controllers

1 Introduction to Spring REST Controllers

In Spring Boot, REST APIs are created using **REST Controllers**.

A REST Controller:

- Handles HTTP requests
- Returns JSON/XML responses
- Does not render HTML views

Spring makes REST a first-class feature using annotations.

2 Creating a REST Controller

2.1 @RestController

Definition

@RestController is used to define a REST controller.

It is a combination of:

@Controller + @ResponseBody

Purpose

- Marks class as a web controller
 - Ensures return values are written directly to HTTP response body
 - Automatically converts objects to JSON
-

Example

@RestController

```
public class UserController {
```

```
@GetMapping("/hello")
public String sayHello() {
    return "Hello World";
}
```

Response:

Hello World

3 Mapping URLs

Spring maps HTTP requests using mapping annotations.

3.1 @RequestMapping

Definition

Used to map:

- URL paths
 - HTTP methods
 - Produces/Consumes content types
-

Example

```
@RequestMapping(value = "/users", method = RequestMethod.GET)
public List<User> getUsers() {
    return userService.findAll();
}
```

3.2 Shortcut Mapping Annotations

Spring provides specific annotations for HTTP methods.

♦ @GetMapping

Handles HTTP GET requests.

`@GetMapping("/users")`

♦ **@PostMapping**

Handles HTTP POST requests.

`@PostMapping("/users")`

♦ **@PutMapping**

Handles HTTP PUT requests.

`@PutMapping("/users/{id}")`

♦ **@DeleteMapping**

Handles HTTP DELETE requests.

`@DeleteMapping("/users/{id}")`

4 Handling Request Parameters

4.1 @RequestParam

Definition

Used to extract query parameters from URL.

Example

Request:

GET /greeting?name=Prasanna

Controller:

```
@GetMapping("/greeting")
public String greet(@RequestParam String name) {
    return "Hello " + name;
}
```

With Default Value

```
@GetMapping("/greeting")
public String greet(@RequestParam(defaultValue = "World") String name) {
    return "Hello " + name;
}
```

If parameter is missing → "World" is used.

4.2 @PathVariable

Definition

Used to extract values from URI path.

Example

Request:

GET /users/10

Controller:

```
@GetMapping("/users/{id}")
public User getUser(@PathVariable Long id) {
    return userService.findById(id);
}
```

10 is assigned to id.

Difference: @RequestParam vs @PathVariable

Feature @RequestParam @PathVariable

Source	Query string	URI path
--------	--------------	----------

Example	?name=John	/users/10
---------	------------	-----------

5 Accepting JSON Data

5.1 @RequestBody

Definition

Used to bind JSON request body to Java object.

Spring automatically converts JSON → Object using Jackson.

Example

Request:

POST /users

Content-Type: application/json

```
{  
  "name": "Prasanna",  
  "email": "prasanna@example.com"  
}
```

Controller

```
@PostMapping("/users")  
public User createUser(@RequestBody User user) {  
    return userService.save(user);  
}
```

Spring:

- Reads JSON
 - Converts to User object
 - Passes to method
-

📁 Complete Example REST Controller

```
@RestController  
@RequestMapping("/users")  
public class UserController {  
  
    @GetMapping("/{id}")
```

```
public User getUser(@PathVariable Long id) {  
    return new User(id, "Prasanna");  
}
```

@PostMapping

```
public User createUser(@RequestBody User user) {  
    return user;  
}  
}
```

7 Internal Flow in REST Controller

1. Request hits Tomcat
2. DispatcherServlet receives request
3. Finds matching controller method
4. Binds parameters (@RequestParam / @PathVariable / @RequestBody)
5. Executes method
6. Converts return object to JSON
7. Sends HTTP response

Interview-Ready Summary

In Spring Boot, REST controllers are created using `@RestController`. URL mappings are defined using `@RequestMapping` and shortcut annotations like `@GetMapping` and `@PostMapping`. Query parameters are handled using `@RequestParam`, path variables using `@PathVariable`, and JSON request bodies using `@RequestBody`. Spring automatically handles JSON conversion (Converting Java objects ↔ JSON format automatically) using Jackson.

1 HTTP METHODS

REST is **resource-based design**.

Example resource:

/api/users

Each HTTP method represents an **action on the resource**.

● GET — Read Data

```
@GetMapping("/api/users")
```

```
public List<UserResponseDto> getAllUsers() {  
    return userService.findAll();  
}
```

🔥 Characteristics:

- Safe ✅ (no modification)
- Idempotent ✅
- Can be cached
- Should NOT modify DB

Interview Question:

Why is GET idempotent?

Because calling it multiple times does not change server state.

● POST — Create Resource

```
@PostMapping("/api/users")
```

```
public ResponseEntity<UserResponseDto> createUser(  
    @RequestBody @Valid UserRequestDto dto) {
```

```
    UserResponseDto created = userService.create(dto);
```

```
    URI location = URI.create("/api/users/" + created.getId());
```

```
return ResponseEntity.created(location).body(created);
}
```

🔥 Characteristics:

- Not idempotent ❌
- Creates new resource
- Returns 201 Created
- Should include Location header

Senior-level best practice:

Return 201 Created + Location header.

🟢 PUT — Full Update

```
@PostMapping("/api/users/{id}")
public ResponseEntity<UserResponseDto> update(
    @PathVariable Long id,
    @RequestBody UserRequestDto dto) {

    return ResponseEntity.ok(userService.update(id, dto));
}
```

🔥 Characteristics:

- Idempotent ✅
 - Replaces full resource
 - If same request repeated → same result
-

🟢 DELETE — Remove Resource

```
@DeleteMapping("/api/users/{id}")
public ResponseEntity<Void> delete(@PathVariable Long id) {
    userService.delete(id);
    return ResponseEntity.noContent().build();
}
```

🔥 Characteristics:

- Idempotent 
 - Return 204 No Content
 - No response body
-

📘 2 Handling Request Parameters & Body

🟡 @RequestParam

Used for query parameters.

GET /api/users?role=ADMIN

@GetMapping("/api/users")

```
public List<UserResponseDto> getByRole(  
    @RequestParam String role) {
```

```
    return userService.findByRole(role);
```

```
}
```

Used for:

- Filtering
 - Sorting
 - Pagination
-

🟡 @PathVariable

Used for resource identification.

GET /api/users/10

@GetMapping("/api/users/{id}")

```
public UserResponseDto getById(@PathVariable Long id) {  
    return userService.findById(id);  
}
```

🟡 @RequestBody

Used for POST/PUT.

What happens internally?

1. Client sends JSON.
2. DispatcherServlet receives request.
3. HttpMessageConverter is triggered.
4. Jackson converts JSON → Java Object.
5. That object is passed into controller.

If JSON invalid?

→ `HttpMessageNotReadableException`

🟢 3 Customizing HTTP Responses

Now the important part.

🟢 ResponseEntity — Full Control

Represents:

Status + Headers + Body

Basic:

```
return ResponseEntity.ok(user);
```

Custom Status:

```
return ResponseEntity.status(HttpStatus.CONFLICT)
    .body("User already exists");
```

Add Header:

```
return ResponseEntity.ok()
    .header("X-Request-Id", "12345")
    .body(user);
```

Created:

```
return ResponseEntity.created(location).body(user);
```

● When NOT to Use ResponseEntity

Simple case:

```
@GetMapping("/hello")
public String hello() {
    return "Hello";
}
```

Spring automatically returns 200.

Cleaner code.

Proper REST Status Code Design Scenario

Status → Success

GET → **200 OK**

POST → **201 Created**

PUT → **200 OK**

DELETE → **204 No Content**

Validation error → **400 Bad Request**

Not found → **404 Not Found**

Conflict → **409 Conflict**

Server error → **500 Internal Server Error**

These are **standard REST conventions**.

Error & Exception Handling in Spring Boot REST

● 1 Why Exception Handling is Important

- ✓ Prevents stack traces from leaking
- ✓ Provides consistent API responses
- ✓ Improves client experience
- ✓ Centralized control
- ✓ Easier debugging & monitoring

2 Standard REST Error Response Format

Professional structure:

```
{  
  "timestamp": "2026-02-19T14:30:00",  
  "status": 404,  
  "error": "Not Found",  
  "message": "User not found with ID: 123",  
  "path": "/users/123"  
}
```

Fields:

- timestamp → when error occurred
- status → HTTP code
- error → reason phrase
- message → human-readable explanation
- path → endpoint

3 Custom Exception

Create domain-specific exceptions.

```
public class ResourceNotFoundException extends RuntimeException {
```

```
    public ResourceNotFoundException(String message) {  
        super(message);  
    }  
}
```

```
}
```

- ✓ Extend RuntimeException
- ✓ Do NOT use generic Exception

4 Throw Exception in Service Layer

```

public User getUserById(Long id) {

    return userRepository.findById(id)
        .orElseThrow(() ->
            new ResourceNotFoundException(
                "User not found with ID: " + id
            ));
}

```

- ✓ Service throws
- ✓ Controller stays clean

5 Global Exception Handling

Use:

`@RestControllerAdvice`

It applies to all controllers.

Example

`@RestControllerAdvice`

```

public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(
        ResourceNotFoundException ex,
        HttpServletRequest request) {

        ErrorResponse error = new ErrorResponse(
            LocalDateTime.now(),
            404,
            "Not Found",

```

```

        ex.getMessage(),
        request.getRequestURI()
    );

    return ResponseEntity.status(HttpStatus.NOT_FOUND)
        .body(error);
}

@ExceptionHandler(Exception.class)
public ResponseEntity<ErrorResponse> handleGeneric(
    Exception ex,
    HttpServletRequest request) {

    ErrorResponse error = new ErrorResponse(
        LocalDateTime.now(),
        500,
        "Internal Server Error",
        "Something went wrong",
        request.getRequestURI()
    );

    return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
        .body(error);
}
}

```

6 ErrorResponse DTO

```
public class ErrorResponse {
```

```
private LocalDateTime timestamp;
private int status;
private String error;
private String message;
private String path;

// constructor + getters
}
```

7 Validation with @Valid

DTO Validation

```
public class UserRequestDto {

    @NotBlank(message = "Name is required")
    private String name;

    @Email(message = "Invalid email")
    private String email;

    @Min(value = 18, message = "Age must be at least 18")
    private int age;
}
```

Use in Controller

```
@PostMapping("/users")
public ResponseEntity<UserResponseDto> create(
    @Valid @RequestBody UserRequestDto dto) {

    return ResponseEntity.ok(service.create(dto));
}
```

```
}
```

If validation fails →

Spring throws:

MethodArgumentNotValidException

8 Handle Validation Exception

```
@ExceptionHandler(MethodArgumentNotValidException.class)
```

```
public ResponseEntity<Map<String, String>> handleValidation(  
    MethodArgumentNotValidException ex) {
```

```
    Map<String, String> errors = new HashMap<>();
```

```
    ex.getBindingResult().getFieldErrors()
```

```
        .forEach(error ->
```

```
            errors.put(error.getField(),
```

```
                error.getDefaultMessage())
```

```
        );
```

```
    return ResponseEntity.badRequest().body(errors);
```

```
}
```

9 Common HTTP Status Codes

Scenario	Status
----------	--------

Success GET	200
-------------	-----

Created	201
---------	-----

Deleted	204
---------	-----

Validation error	400
------------------	-----

Not found	404
-----------	-----

Scenario	Status
Conflict	409
Forbidden	403
Server error	500

API Documentation – Swagger / OpenAPI (Spring Boot)

1 Why API Documentation?

Without documentation:

- ✗ Frontend doesn't know endpoints
- ✗ Other teams don't know request/response format
- ✗ No clear contract

Solution → **OpenAPI (Swagger)**

2 What is OpenAPI?

OpenAPI = Standard specification for describing REST APIs.

It describes:

- Endpoints
 - Request parameters
 - Request body
 - Response body
 - Status codes
 - Authentication
 - Models (schemas)
-

3 What is Swagger?

Swagger UI = Tool that:

- Reads OpenAPI spec
- Shows interactive API documentation

- Allows “Try it out” feature
-

4 Library Used in Spring Boot 3

Use:

springdoc-openapi-starter-webmvc-ui

This automatically:

- ✓ Generates OpenAPI JSON
 - ✓ Generates Swagger UI
 - ✓ Scans controllers
-

5 Default URLs

After adding dependency:

OpenAPI JSON:

/v3/api-docs

Swagger UI:

/swagger-ui/index.html

6 Customizing Path

Change API docs path:

springdoc.api-docs.path=/api-docs

Change Swagger UI path:

springdoc.swagger-ui.path=/swagger-ui.html

7 Basic Automatic Documentation

Spring automatically documents:

- `@RestController`
- `@GetMapping`, `@PostMapping`, etc.
- `@RequestParam`
- `@PathVariable`
- `@RequestBody`

- @ResponseStatus

Even without writing anything extra.

Spring Security:

Problem Without JWT (Traditional Approach)

Earlier systems used:

- Sessions
- Cookies

 Flow:

1. User logs in
2. Server creates session
3. Stores session in memory/database
4. Every request checks session

Problems:

- Not scalable (server stores state)
 - Hard for microservices
 - Performance issues
-

Solution: JWT (Stateless Authentication)

 With JWT:

- No session stored in server
 - Everything stored in **token**
-

JWT Concept (Very Simple)

When user logs in:

1. Server verifies credentials
2. Server generates **JWT token**

3. Sends token to client
 4. Client sends token in every request
-

Flow with JWT

Login → Get Token → Send Token in Headers → Access APIs

Example:

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6...

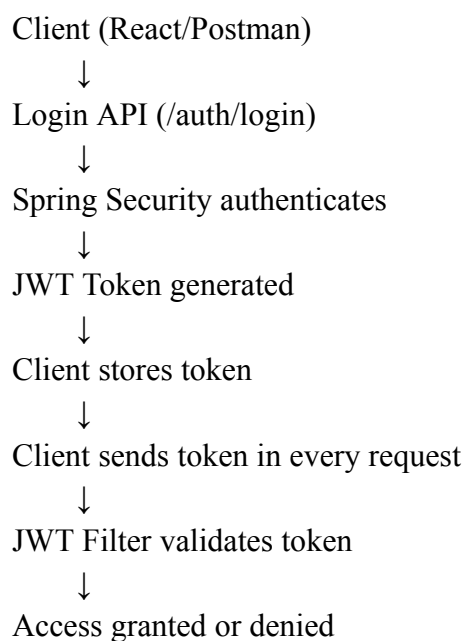
What Spring Security + JWT Together Do

- Spring Security → Handles security framework
- JWT → Handles stateless authentication

 Together:

- Secure APIs
 - No sessions
 - Role-based access
-

High-Level Architecture (VERY IMPORTANT FOR INTERVIEWS)



Golden Rule (VERY IMPORTANT)

Always remember:

👉 **JWT = Identity proof**

👉 **Spring Security = Security manager**

1. Introduction to Spring Security

What is Spring Security?

Spring Security is a powerful and widely used framework in the Spring ecosystem that provides **authentication and authorization mechanisms** to secure applications.

It helps developers protect applications from unauthorized access by offering **pre-built security features** such as login handling, role-based access control, and protection against common security threats.

Technically, Spring Security works using a **filter chain mechanism**, where every incoming HTTP request passes through a series of security filters before reaching the application.

Key Features of Spring Security

- Provides authentication (verifying user identity)
 - Provides authorization (controlling access to resources)
 - Supports multiple authentication methods (Basic, JWT, OAuth, etc.)
 - Protects REST APIs and web applications
 - Easily configurable and extendable
 - Works seamlessly with Spring Boot
-

Why Do We Need to Secure REST APIs?

REST APIs expose application data over HTTP. Without proper security, they are vulnerable to misuse.

Reasons to Secure APIs:

- Prevent unauthorized access to sensitive data
- Protect user information (credentials, personal data)
- Avoid malicious attacks (data theft, unauthorized actions)
- Ensure only authenticated users can access protected endpoints
- Maintain data integrity and confidentiality

⚠ Example:

Without security:

GET /api/users

→ Anyone can access all user data ❌

With security:

→ Only authorized users can access ✅

🔑 Authentication vs Authorization

♦ Authentication (WHO ARE YOU?)

Authentication is the process of verifying the identity of a user.

It ensures that the user is who they claim to be.

Example:

- Login using username and password
- OTP verification

User → enters credentials → system verifies → access granted/denied

♦ Authorization (WHAT CAN YOU DO?)

Authorization is the process of determining what actions a user is allowed to perform after authentication.

It ensures users can only access resources they are permitted to.

Example:

- Admin can delete users
 - Normal user can only view profile
-

📊 Difference Between Authentication and Authorization

Feature	Authentication	Authorization
Purpose	Verify identity	Check permissions
When	Before access	After authentication
Example	Login	Role-based access

HTTP Basic Authentication vs Bearer Token Authentication

♦ HTTP Basic Authentication

A simple authentication method where credentials are sent in every request.

Authorization: Basic base64(username:password)

Limitations:

- Credentials sent repeatedly
- Not secure for production systems
- No scalability
- Vulnerable to attacks if not used with HTTPS

♦ Bearer Token Authentication (JWT)

A modern and secure method where a token is used instead of credentials.

Authorization: Bearer <JWT_TOKEN>

Flow:

1. User logs in with credentials
2. Server generates a JWT token
3. Client stores the token
4. Client sends token in every request
5. Server validates token and allows/denies access

Advantages of Bearer Token (JWT)

- Stateless (no session storage on server)
- Scalable for distributed systems
- More secure than Basic Authentication
- Suitable for REST APIs and microservices

Summary

- Spring Security is used to secure applications by handling authentication and authorization.
 - Authentication verifies identity, while authorization controls access.
 - REST APIs must be secured to prevent unauthorized access and protect data.
 - HTTP Basic Authentication is outdated and less secure.
 - JWT-based Bearer Token Authentication is modern, secure, and widely used in real-world applications.
-

Interview Answer (Short)

Spring Security is a framework that provides authentication and authorization to secure applications. It uses a filter-based approach to intercept requests. Authentication verifies user identity, while authorization controls access based on roles. For REST APIs, JWT-based authentication is preferred because it is stateless, secure, and scalable.

2. Introduction to JWT (JSON Web Token)

What is JWT?

JSON Web Token (JWT) is an **open standard (RFC 7519)** used to securely transmit information between parties as a **compact, self-contained JSON object**

It is commonly used for:

- Authentication (user identity)
- Authorization (access control)

JWT is **digitally signed**, so it can be verified and trusted.

Structure of JWT

A JWT consists of **three parts separated by dots (.)**:

Header.Payload.Signature

Example:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpzZW50a3kiLCJ1d2UiOiJxMjM0NTYifQ.signature

♦ 1. Header

Contains metadata about the token:

- Type of token (JWT)
- Signing algorithm (e.g., HS256)

Example:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

♦ 2. Payload

Contains **claims (data)** about the user.

Types of claims:

- Registered (iss, exp, sub, etc.)
- Public
- Private (custom data)

Example:

```
{
  "sub": "1234567890",
  "name": "Sweety",
  "role": "USER"
}
```

⚠ Note: Payload is **readable (not encrypted)**, so do not store sensitive data.

♦ 3. Signature

Used to verify that the token is **not tampered**.

Created using:

```
HMACSHA256(
  base64(header) + "." + base64(payload),
  secret_key
)
```

👉 Ensures:

- Data integrity
- Token authenticity

Key Features of JWT

♦ Stateless

- No session stored on server
- All information is inside token

♦ Compact

- Small size → easy to send in HTTP headers

♦ Self-contained

- Contains user info + permissions
- No need to query DB every time

👉 These features improve **performance and scalability**

JWT Authentication Flow

Step-by-Step Process:

1. User logs in with credentials
 2. Server validates credentials
 3. Server generates JWT token
 4. Token sent to client
 5. Client stores token
 6. Client sends token in every request
 7. Server validates token
 8. Access granted or denied
-

Flow Representation

Login → Token Generated → Client Stores Token → Sends Token → Server Validates → Access

Request Example

Authorization: Bearer <JWT_TOKEN>

Why JWT is Used in API Security

- Eliminates need for server-side sessions
 - Reduces server load
 - Enables scalability (microservices)
 - Secure communication using signed tokens
 - Easy integration with frontend (React, Angular, etc.)
-

Important Points

- JWT is **Base64 encoded**, not encrypted
 - Signature ensures token is not modified
 - Token can have expiration (exp)
 - Used widely in modern REST APIs
-

Interview Answer (Short)

JWT is a compact, self-contained token used for authentication and authorization. It consists of three parts: header, payload, and signature. After successful login, the server generates a JWT and sends it to the client. The client includes the token in the Authorization header for subsequent requests. The server validates the token to grant access. JWT is stateless, scalable, and widely used in REST APIs.

3. Generating and Validating JWT

Overview

JWT generation and validation are core parts of authentication in Spring Security.

We create a **JWT Utility Class** that handles:

- Token generation
- Token validation
- Extracting user details from token

This is usually implemented using the **io.jsonwebtoken (jjwt) library**

Required Dependency (JJWT)

Add the following dependencies:

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.11.5</version>
</dependency>
```

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.11.5</version>
</dependency>
```

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.11.5</version>
</dependency>
```

JWT Utility Class (JwtService)

This class is responsible for:

- Creating tokens
- Extracting data
- Validating tokens

♦ Complete JWT Utility Class

```
package com.example.security;
```

```
import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.io.Decoders;
import io.jsonwebtoken.security.Keys;
```

```
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Component;
```

```

import java.security.Key;
import java.util.Date;
import java.util.function.Function;
import java.util.HashMap;
import java.util.Map;

@Component
public class JwtService {

    // Secret key (Base64 encoded)
    private static final String SECRET_KEY = "your_secret_key_here_base64";

    // ♦ Generate Token
    public String generateToken(String username) {
        Map<String, Object> claims = new HashMap<>();
        return createToken(claims, username);
    }

    // ♦ Create Token
    private String createToken(Map<String, Object> claims, String username) {
        return Jwts.builder()
            .setClaims(claims)
            .setSubject(username) // username stored in payload
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + 1000 * 60 * 30)) // 30 min
            .signWith(getSignKey(), SignatureAlgorithm.HS256)
            .compact();
    }

    // ♦ Extract Username from Token
    public String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }

    // ♦ Extract Expiration
    public Date extractExpiration(String token) {
        return extractClaim(token, Claims::getExpiration);
    }

    // ♦ Extract Specific Claim
    public <T> T extractClaim(String token, Function<Claims, T> resolver) {
        final Claims claims = extractAllClaims(token);
        return resolver.apply(claims);
    }

```

```

    }

    // ♦ Extract All Claims
    private Claims extractAllClaims(String token) {
        return Jwts.parserBuilder()
            .setSigningKey(getSignKey())
            .build()
            .parseClaimsJws(token)
            .getBody();
    }

    // ♦ Check Token Expiry
    private Boolean isTokenExpired(String token) {
        return extractExpiration(token).before(new Date());
    }

    // ♦ Validate Token
    public Boolean validateToken(String token, UserDetails userDetails) {
        final String username = extractUsername(token);
        return (username.equals(userDetails.getUsername()) && !isTokenExpired(token));
    }

    // ♦ Get Signing Key
    private Key getSignKey() {
        byte[] keyBytes = Decoders.BASE64.decode(SECRET_KEY);
        return Keys.hmacShaKeyFor(keyBytes);
    }
}

```

Key Methods Explained

♦ 1. generateToken()

- Creates JWT after login
 - Stores username in payload
-

♦ 2. extractUsername()

- Extracts username from token payload
-

♦ 3. validateToken()

- Checks:
 - Username matches
 - Token is not expired
-

♦ 4. extractClaim()

- Generic method to extract any data from JWT
-

Flow of JWT Utility

Login → generateToken() → Token Created

Request → extractUsername() → validateToken() → Access Granted

7. Securing REST APIs with JWT Bearer Token

Overview

We secure APIs by:

- Making some endpoints **public**
 - Making others **protected**
 - Validating JWT in every request
-

Flow

Login → Get JWT → Send Token → Validate → Access Protected API

Endpoint Design

♦ Public APIs

/api/auth/**

- Login, register
 - No token required
-

♦ Protected APIs

/api/users/**

- Requires valid JWT
 - Access controlled
-

★ IMPORTANT CONFIG (ACCESS RULES)

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/auth/**").permitAll()
    .requestMatchers("/api/users/**").authenticated()
)
```

🌐 Request Example (Protected API)

GET /api/users/profile

Authorization: Bearer <JWT_TOKEN>

🧠 What Happens Internally

1. Request comes with JWT
 2. JWT Filter extracts token
 3. Token is validated
 4. User is authenticated
 5. Access granted
-

⚠ Without Token

- Request is rejected ❌
 - 401 Unauthorized
-

🎯 Steps to Use JWT

1. Call /authenticate with username & password
2. Receive JWT token
3. Add token in header:

Authorization: Bearer <JWT>

4. Access protected APIs

Interview Answer

Authentication controller validates user credentials and generates a JWT upon successful login. Protected APIs require the token in the Authorization header using the Bearer scheme. The JWT filter validates the token for each request, ensuring only authenticated users can access secured endpoints.